NutriSense

Recommended Foods — Logic & Technical Details

How the Assessment page builds the Recommended foods list: which nutrients get a section, how foods are ranked, how diet settings filter them, and how this differs from the Random Forest model.

Not machine learning · Rule-based lookup on food_items · Code: app/services/recommendation_service.py · API: GET /api/recommendations

1. Simple picture

The Random Forest only answers: “How likely is this person deficient in each nutrient?” It does not suggest foods.

Recommended foods is a separate, rule-based step. In plain language:

1. Look at the assessment risks.
2. Keep every nutrient whose probability is 45% or higher.
3. For each of those nutrients, search the food catalog.
4. Drop foods that break the user’s diet settings.
5. Keep the 6 remaining foods with the most of that nutrient per 100 g.
6. Show one card per nutrient on the Assessment page.

The meal planner reuses the same nutrient list and the same diet filters. It is also rule-based, not ML.

2. End-to-end flow

Step 1 — Assessment already exists

The user must have run an assessment. Risks are stored as JSON on the assessments row (risks_json). Each risk has a nutrient key and a probability between 0 and 1 (from predict_proba, then optional lab boost).

Step 2 — Frontend asks for recommendations

Assessment.jsx calls GET /api/recommendations?assessment_id=<id>. If assessment_id is omitted, the backend uses the user’s latest assessment.

Step 3 — Choose which nutrients to recommend for

high_risk_nutrients() keeps every nutrient where probability >= 0.45 and the nutrient has a matching food_items column. Order is the same as in risks_json (highest risk first in the UI).

```
cutoff = 0.45   (45%)
keep nutrient if  probability >= 0.45
and nutrient is one of: iron, vitamin_d, vitamin_b12, calcium,
  folate, vitamin_a, vitamin_c, magnesium, zinc, protein
```

Earlier versions kept only the top 3 nutrients. Now every nutrient above the cutoff gets its own food list.

Step 4 — Load diet settings from the health profile

diet_restrictions is a JSON list on health_profiles (the same chips the user set in onboarding / Profile). diet_flags() turns that list into booleans.

Step 5 — Query food_items per nutrient

For each high-risk nutrient, get_recommendations() maps the nutrient name to a column (for example calcium → calcium_mg). It sorts the catalog so preferred sources come first, then the highest value of that column.

Step 6 — Filter, then take 6 foods

SQL already filters vegan / vegetarian flags when those diets are on. Then Python drops USDA noise names and applies keyword filters (gluten-free, dairy-free, nut allergy). The first 6 foods that survive are returned for that nutrient.

Step 7 — UI grouping

The API returns a list of blocks: { nutrient, foods[] }. The Assessment page renders one card per block. The number shown next to each food is that nutrient's stored value (per 100 g), not a combined Fe · Ca line.

3. Nutrient → food column mapping

Foods are ranked by the amount stored on food_items. Those amounts are always per 100 grams. serving_gram_weight is only a default portion for the diary UI; recommendations do not rescale to a serving.

Risk nutrient (from assessment)	Column used to rank foods	UI label example
iron	iron_mg	Fe 8.5
calcium	calcium_mg	Ca 975
vitamin_b12	vitamin_b12_mcg	B12 0
folate	folate_mcg	Folate 146
vitamin_a	vitamin_a_mcg	Vit A 469
vitamin_c	vitamin_c_mg	Vit C 59
magnesium	magnesium_mg	Mg 392
zinc	zinc_mg	Zn 5.8
vitamin_d	vitamin_d_mcg	Vit D 0
protein	protein_g	Protein 21g

Ranking for one nutrient (calcium example):

```
ORDER BY source_priority ASC, calcium_mg DESC
then keep first 6 foods that pass diet filters
```

4. Which catalog foods are preferred

The same food_items table holds three kinds of rows. Recommendations prefer Indian / curated names over generic USDA rows when nutrient values are otherwise similar, because source_priority is the first sort key.

data_type	Source	Sort rank
curated	data/seed_foods.json (familiar Indian names)	0 — first
ifct2017	IFCT 2017 (~542 Indian foods, NIN Hyderabad)	1 — second
anything else (USDA, etc.)	USDA FoodData Central fallback	2 — last

After SQL returns candidates, names matching USDA_SKIP are dropped (babyfood, candies, snacks, pie, puddings, salad dressing, refrigerated dough, food distribution program, artificial flavor). This is is_usda_noise() in app/utils.py.

5. Diet filters

Profile chips are stored as strings in diet_restrictions. Matching is case-insensitive.

Profile chip	Flag set	How foods are filtered
vegan	vegan=True, vegetarian=True, dairy_free=True	Keep only is_vegan = 1. Dairy keywords also blocked.
vegetarian	vegetarian=True (unless already vegan)	Keep only is_vegetarian = 1. Vegan is a stricter subset.
gluten-free	gluten_free=True	Drop if name / ingredients / tags contain gluten keywords.
dairy-free	dairy_free=True	Drop if text contains dairy keywords. Also implied by vegan.
nut allergy	nut_allergy=True	Drop if text contains nut keywords.

Keyword lists (substring match on lowercased name + ingredients + tags):

Filter	Keywords
Gluten	wheat, gluten, bread, atta, maida, seitan, barley, rye, pasta, noodle, biscuit, chapati, roti, paratha, semolina, vermicelli, bulgur, cracker
Dairy	milk, cheese, yogurt, yoghurt, curd, dahi, paneer, butter, ghee, cream, whey, casein, lassi
Nuts	almond, peanut, cashew, walnut, pistachio, hazelnut, pecan, groundnut, tree nut, nuts, nut

Vegan / vegetarian use database flags (is_vegan, is_vegetarian) in SQL first, then food_allowed() repeats those checks plus the keyword filters. Keyword filters are heuristic: they can over-block (for example “nut ” in an unrelated name) or under-block (a gluten food whose name has none of the keywords).

Candidate pool size before the final cut of 6:

```
limit = 6 foods shown per nutrient (Assessment UI)
SQL fetches max(limit × 8, 48) = 48 candidates
then Python filters noise + diet  →  first 6 kept
```

Meal plan generation calls the same function with limit=50 so it has a larger diet-safe catalog to pick meals from.

6. API contract

Item	Detail
Endpoint	GET /api/recommendations
Auth	Logged-in user (JWT)
Query	assessment_id (optional). Must belong to the current user.
404	No assessment yet — "Run an assessment first"
Response	list of RecommendationOut: { nutrient, foods: FoodOut[] }

Example shape (shortened):

```
[
  {
    "nutrient": "calcium",
    "foods": [
      { "id": 12, "display_name": "Sesame seeds (til)",
        "calcium_mg": 975, "is_vegan": true, ... }
    ]
  },
  { "nutrient": "vitamin_b12", "foods": [ ... ] }
]
```

7. Worked example (vegan + gluten-free + nut allergy)

User munikiran@gmail.com. Assessment wellness score 36%. Several nutrients are HIGH (probability well above 0.45), so each gets a recommendation card.

Calcium works as expected: sesame seeds, firm tofu, ragi flour, flaxseed, spinach, roasted chana — all vegan, no gluten keywords, no nut keywords, sorted by calcium_mg per 100 g.

Vitamin B12 looks wrong at first glance (spinach B12 0, masoor dal B12 0). That is the rules working, not a ranking bug:

What happens	Why
True B12 foods are mostly animal (fish, eggs, dairy, fortified products)	Vegan flag drops is_vegan = 0 rows
Plant foods naturally have ~0 mcg B12	Catalog values are honest
SQL still sorts remaining vegan rows by vitamin_b12_mcg DESC	Many ties at 0, so other vegan foods fill the list
UI prints B12 0	It shows that nutrient's column, not iron or calcium

In short: the engine recommends “highest B12 among foods this diet allows.” If the diet allows none with real B12, the list can be empty of useful B12 or filled with zeros. A supplement / fortified-food note is not currently generated.

8. What this is not

Common assumption	Actual behaviour
The Random Forest picks the foods	No. Model only produces risk probabilities.
Foods are scaled to a typical serving	No. Values are per 100 g as stored.
Only the 3 worst nutrients are shown	No. Every nutrient with P ≥ 0.45.
Lists ignore diet settings	No. Vegan/veg flags + keyword filters apply.
B12 0 means a data error	Often means diet filters removed animal B12 sources.
Meal plan is a second ML model	No. Same high-risk list + same filters, then slot-picking rules.

9. Code map

File	Role
backend/app/routers/recommendations.py	GET /api/recommendations — load assessment, call service
backend/app/services/recommendation_service.py	high_risk_nutrients, diet_flags, food_allowed, get_recommendations
backend/app/utils.py	is_usda_noise, food_to_schema (display names)
backend/app/routers/meal_plan.py	Reuses high_risk_nutrients + get_recommendations(limit=50)
backend/app/services/meal_planner.py	Picks meals from that diet-safe catalog
frontend/src/pages/Assessment.jsx	Fetches recs, one card per nutrient, shows the matching nutrient stat

10. One-line summary

```
INPUT:   assessment.risks_json  +  health_profiles.diet_restrictions
KEEP:    nutrients with probability ≥ 0.45
RANK:    food_items by (curated → IFCT → USDA), then that nutrient per 100 g
FILTER:  vegan / vegetarian flags, gluten / dairy / nut keywords, USDA noise
OUTPUT:  up to 6 foods per high-risk nutrient  (Assessment UI)
NOT ML:  Random Forest is not used in this step
```

Educational / internship demonstration only. Not a medical device and not clinical advice. Diet keyword filters are approximate and not a substitute for allergen labelling.